

ESPECIFICACIÓN AMPLIADA DE APIS E INTEGRACIONES

Documento de Integración de Servicios y APIs

Justificación Estratégica, Rutas de Mapas Libres, Mensajería e Hilo de Datos en Vivo — SEDA v1.0.0

Integrantes del Equipo:	• Martinez Morales David • Lopez Beristain Daniel • Garcia Sanchez Nestor Antoine • Medina Cobos Elian • Bazan Aguila Maria Jose • Bañuelos Cano Carlos Gabriel
Grupo Académico:	9A
Organización:	SEDA
Ecosistema Tecnológico:	React / Node.js / SQLite3 / Firebase / OSRM / Infobip

Tabla de Contenido

1. Arquitectura y Justificación de Tecnologías Externas	3
2. Módulo de Rutas y Mapas: OpenStreetMap + OSRM	4
3. Mensajería en Tiempo Real y Alertas: Firebase (Firestore + FCM)	5
4. Canal de Notificaciones de Respaldo: API de Infobip	6
5. Estructura de Endpoints del Backend Local de SEDA	6

1. Arquitectura y Justificación de Tecnologías Externas

El desarrollo e implementación del ecosistema SEDA exige soluciones robustas que garanticen la máxima disponibilidad del servicio en condiciones operativas adversas (como zonas rurales o de alta marginación) sin comprometer la viabilidad financiera del proyecto. A continuación, se detalla la justificación técnica de la selección de cada proveedor e integración externa:

1.1. OpenStreetMap + OSRM (Rutas y Mapas Libres)

La selección de este stack frente a opciones comerciales del mercado como Google Maps API o Mapbox se fundamenta en:

- **Viabilidad Financiera:** SEDA es una iniciativa con enfoque social. Las APIs comerciales cobran tarifas bajo esquemas rígidos de pago por uso (pay-as-you-go). Implementar OpenStreetMap y OSRM garantiza una operación 100% gratuita y sin límites de peticiones.
- **Rendimiento de Enrutamiento:** OSRM procesa grafos viales utilizando optimizaciones basadas en jerarquías de contracción (Contraction Hierarchies). Esto permite resolver algoritmos de ruta óptima en milisegundos directamente en el servidor sin latencias externas de red.

1.2. Firebase (Firestore + Cloud Messaging)

El canal interactivo del personal requiere persistencia inmediata e hilos de eventos asíncronos distribuidos:

- **Sincronización Interactiva Directa:** Firestore opera mediante WebSockets reactivos en lugar de peticiones repetitivas (HTTP polling), reduciendo drásticamente la sobrecarga de datos y transfiriendo las actualizaciones en milisegundos a las aplicaciones activas.
- **Modo Offline Integrado:** Los repartidores transitan por zonas de baja conectividad. Firestore permite almacenar localmente las mutaciones de datos del chat de campo y sincronizarlas de forma automática y transparente una vez restablecido el enlace celular.
- **Despertar de Aplicación (Push Notifications):** Firebase Cloud Messaging (FCM) permite notificar alertas críticas de cadena de frío y reasignaciones urgentes incluso cuando el dispositivo móvil del repartidor se encuentra bloqueado o con la aplicación en segundo plano.

1.3. Infobip (Redundancia Celular SMS)

La infraestructura basada puramente en redes de datos móviles presenta serias deficiencias en el entorno físico de las comunidades de destino (clasificadas como "Muy Malo"):

- **Conmutación Automática por Pérdida de Datos:** Cuando el sistema detecta que la aplicación móvil pierde conectividad IP, la lógica del backend conmuta la entrega de alertas hacia la red celular básica por medio de canales de voz/mensajería SMS transaccionales.
- **Inclusión y Sencillez en Destino:** Los coordinadores comunitarios locales no siempre poseen smartphones con conectividad de alta velocidad. El envío masivo de notificaciones SMS a través de Infobip asegura que cualquier teléfono celular convencional reciba las especificaciones de la canasta sin depender de una red de datos.

2. Módulo de Rutas y Mapas: OpenStreetMap + OSRM

Esta integración provee las coordenadas viales exactas y el trazado geométrico necesario para desplegar en la interfaz del repartidor el camino más corto hacia la comunidad asignada.

2.1. Consulta de Ruta Óptima de Reparto

Este servicio obtiene la geometría del trayecto terrestre idóneo conectando el almacén central con las coordenadas específicas de la comunidad de destino.

Endpoint OSRM: `https://router.project-osrm.org/route/v1/driving/{lng_origen},{lat_origen};{lng_destino},{lat_destino}`

Método: GET

Ejemplo de Respuesta (JSON):

```
{
  "code": "Ok",
  "routes": [
    {
      "geometry": "}_{pyF`_vG_g@`_B~f@...",
      "legs": [
        {
          "distance": 8432.5,
          "duration": 542.8,
          "summary": "Avenida de los Insurgentes, Calle 4"
        }
      ],
      "distance": 8432.5,
      "duration": 542.8
    }
  ],
  "waypoints": [
    { "name": "Almacén Central SEDA" },
    { "name": "Comunidad 12" }
  ]
}
```

3. Mensajería en Tiempo Real y Alertas: Firebase

Permite mantener en comunicación reactiva constante al empacador dentro del almacén y al chofer en ruta.

3.1. Estructura de Datos en Firestore (Canales de Chat)

El canal de comunicación se orquesta mediante una colección persistente en NoSQL dentro de `/chats/{id_ruta}/messages` con el siguiente esquema JSON:

```
{
  "senderId": "usr_repartidor_02",
  "senderName": "Daniel Lopez",
  "messageText": "Llegando a la comunidad, la vía principal está obstruida.",
}
```

```
"timestamp": "2026-07-16T14:59:12.432Z",  
"alertLevel": "high"  
}
```

3.2. Notificaciones Push a Dispositivos Móviles (FCM)

Endpoint FCM: <https://fcm.googleapis.com/v1/projects/seda-sistema/messages:send>

Método: **POST**

Payload de Entrada (JSON):

```
{  
  "message": {  
    "topic": "comunidades_criticas",  
    "notification": {  
      "title": "¡Ruta Prioritaria Asignada!",  
      "body": "Se ha asignado un nuevo despacho urgente a la Comunidad 15 con estatus Muy Malo."  
    },  
    "data": {  
      "rutaId": "rt_9872",  
      "prioridad": "Muy Malo"  
    }  
  }  
}
```

4. Canal de Notificaciones de Respaldo: API de Infobip

Pasarela transaccional utilizada como contingencia cuando las conexiones persistentes de Firebase pierden respuesta IP por falta de datos celulares en campo.

Endpoint Infobip: `https://api.infobip.com/sms/2/text/advanced`

Método: **POST**

Headers Obligatorios:

Authorization: App {API_KEY_INFOBIP}

Content-Type: application/json

Payload de Entrada (JSON):

```
{
  "messages": [
    {
      "destinations": [
        { "to": "522712345678" }
      ],
      "from": "SEDA_Alert",
      "text": "SEDA Alerta: El cargamento de despensas va en ruta y llegará en 20 minutos. Favor de preparar recepción"
    }
  ]
}
```

5. Estructura de Endpoints del Backend Local de SEDA

APIs locales del servidor que gestionan el flujo transaccional y persisten en la base de datos centralizada SQLite3.

5.1. Registrar Entrada de Donación

Endpoint: `/api/donaciones`

Método: **POST**

Payload de Entrada:

```
{
  "donante": "Walmart",
  "categoria": "Lácteos",
  "peso_kg": 340.50,
  "temperatura_entrada": 3.4
}
```

Respuesta Exitosa (201 Created):

```
{
  "status": "success",
  "message": "Donación registrada correctamente e ingresada en almacén.",
}
```

```
"data": {
  "lote_id": "LOTE-2026-0034",
  "fecha_registro": "2026-07-16T15:00:00Z",
  "kanban_estado": "PENDIENTE"
}
```

5.2. Actualizar Estado de Ruta (Campo / Delivery)

Endpoint: `/api/rutas/{ruta_id}/status`

Método: **PUT**

Payload de Entrada:

```
{
  "nuevo_estado": "Entregado",
  "comentarios": "Recibido por el comité local sin incidencias técnicas.",
  "firma_receptor": "Firma_Bazan_MJ"
}
```

Respuesta Exitosa (200 OK):

```
{
  "status": "success",
  "message": "Estado de ruta actualizado a Entregado.",
  "data": {
    "ruta_id": "rt_9872",
    "actualizado_el": "2026-07-16T15:02:12Z"
  }
}
```